

AI Report

AI tool used. OpenAI Codex.

Example prompts (edited for clarity).

1. Create a `TODO.md` to plan the implementation, including test cases. Follow a TDD workflow: write failing unit tests before implementing the code, then write the minimum code needed to pass them. For now, only create the plan; do not begin implementation.
2. My backend is disorganized. Refactor it into a clear MVC structure, with separate folders under `src/` and clearly defined configuration. Group schema registration and data ingestion into one view, and dashboard registration and dashboard data display into another.
3. Organize tests to mirror the `src/` structure. Separate unit tests from end-to-end and integration tests for readability.
4. Treat the instructions in `AGENTS.md` as part of the prompt.

Accepted suggestion. I accepted the AI suggestion to keep backend data in memory. I initially considered file storage, but the assignment requires in-memory storage and no persistence. File storage would add complexity without helping meet the requirements. I also reconsidered my plan for a more elaborate test structure around every route. That level of organization might suit a larger project, but felt unnecessary for this assignment. AI helped me recognize where I was overcomplicating the solution and stay focused on the requirements.

Rejected approach. AI initially suggested keeping the backend in a single file because it was only around 100-200 lines. I preferred an MVC-style structure, separating in-memory state into models, business logic into controllers, and HTTP handling into routes. I also grouped shared HTTP helpers under `routes/common/` so their purpose was clear from their location. Similarly, I rejected placing all tests in one folder. Mirroring the source structure and separating unit tests from integration tests makes it easier to see what each test covers. For me, this was a balance between keeping the project small and making it easy for both people and AI to navigate.

Validation. I used a TDD workflow, writing failing regression tests before changing behavior. Unit and integration tests check the expected behavior and provide repeatable feedback. Manual API checks can complement these tests using Postman, curl, or FastAPI's Swagger UI. My `AGENTS.md` also instructs the AI to run `make test` and `make lint` after backend changes, checking behavior and code consistency. This gives the AI a deterministic way to check its output, while I remain responsible for reviewing the code and whether the tests cover the requirements.